

I read the diagram against the agent implementation in [legal_workflow_generator/agent/](#). Short version: **the diagram is mostly not implemented**. The code is a single-pass self-corrective LangGraph loop that fakes multi-domain support by concatenating all domains into one retrieve+generate call. The diagram's defining ideas — decomposition, per-domain isolated subroutines, and a stitching layer — are absent.

Element-by-element

Diagram element	Status	Where / why
User query	□	<code>graph.invoke({"query": ...})</code>
Query processing unit (Phase 2)	□	nodes.py:37 <code>classify_query</code> → normalizer + intent classifier + context resolver
Decomposition layer (split into sub-tasks, max 3 domains)	× partial	<code>classify_query</code> makes an LLM call to detect <code>all_domains</code> (nodes.py:53-71), but it does not split into independent sub-tasks and does not cap at 3 (can return all 5)
Sub-task: domain A / B / C (parallel branches)	×	No fan-out. One linear graph, one state object.
Per-sub-task subroutine, once per domain	×	retrieve loops domains <i>inside a single node</i> and merges everything into one pool (<code>sorted(...)[:7]</code>). Every downstream node works on that merged pool, not per domain.
Transform query	⚠	<code>rewrite_query</code> node exists, but it's global, not per-domain; <code>retrieve</code> also bolts on static hint strings per domain
Retrieve	□	<code>_rag_pipeline.searcher.search</code>
Grade context — cap 3 retries, same domain	□ cap / × "same domain"	<code>route_after_context_grade</code> abstains at <code>retry_count_retrieval >= 3</code> (graph.py:24); retry re-runs across all domains, not scoped
Generate — answer + inline citations	□ / ⚠	<code>generate</code> node; "citations" are just the retrieved <code>provision_id</code> s dumped in a SOURCES block — not per-

Diagram element	Status	Where / why
		claim inline citations
Verify citations — Section refs exist in chunks	⚠ partial	<code>verify_citations</code> (nodes.py:223) string-matches citation ids against retrieved doc ids → <code>verified</code> / <code>failed</code> lists
"fabricated → strip claim, regenerate it, retries exhausted"	×	<code>failed_citations</code> is written and then nothing reads it . No claim-stripping, no regenerate trigger, no edge from <code>verify_citations</code> back to <code>generate</code> .
Grade groundedness — cap 2 retries	□ / ⚠	<code>route_after_groundedness</code> regenerates until <code>retry_count_generation</code> ≥ 2 — but <code>route_after_verify_citations</code> (graph.py:49) skips groundedness entirely whenever <code>len(all_domains) > 1</code> , i.e. exactly the multi-domain case
Grade answerable	□	<code>grade_answerability</code> ; on <code>off_target</code> it loops back to <code>rewrite_query</code> reusing the retrieval counter
Sub-task answered / abstained	⚠	Whole-query <code>abstain</code> node, not per-sub-task — one abstain kills the entire answer
Stitching layer (order steps, merge citations, flag gaps)	×	Does not exist. The single <code>generate</code> prompt is just told to "group by domain"; no post-hoc ordering, cross-subtask citation merge, or gap flagging.
Final workflow + citations	□	<code>result["answer"]</code> + <code>result["verified_citations"]</code>

What's actually there

The middle "per-sub-task subroutine" box, taken alone, is a fair description of the current graph:
 retrieve → grade context → (rewrite ↔ retrieve) → generate → verify → grade groundedness →
 (regenerate ↔ generate) → grade answerable → answered/abstain. That self-correction loop is
 implemented. It just runs **once, globally**, not N times in isolation per domain.

Gaps to close if you want the diagram

1. **Decomposition layer** — new node that splits into ≤ 3 `SubTask` objects (domain + scoped sub-question).
2. **Fan-out** — `LangGraph` `send` / subgraph so the subroutine runs per sub-task with its own retry counters and its own doc pool.
3. **Citation-failure path** — wire `failed_citations` to a strip-and-regenerate step; currently dead data.
4. **Stitching layer** — new terminal node to merge per-subtask answers, dedupe/merge citations, order steps, and flag domains that abstained as "gaps."
5. **Per-subtask abstain** — abstain should mark one sub-task, not the whole response.
6. Remove the multi-domain groundedness bypass (Known Limitation #3 in [PHASE3_EVAL_RESULTS.md](#)) — the fan-out makes it unnecessary.

Note [docs/architecture.png](#) in the repo is a different, older block diagram

(Query/Retrieval/Workflow/Response units) — it doesn't match the diagram you posted either.